

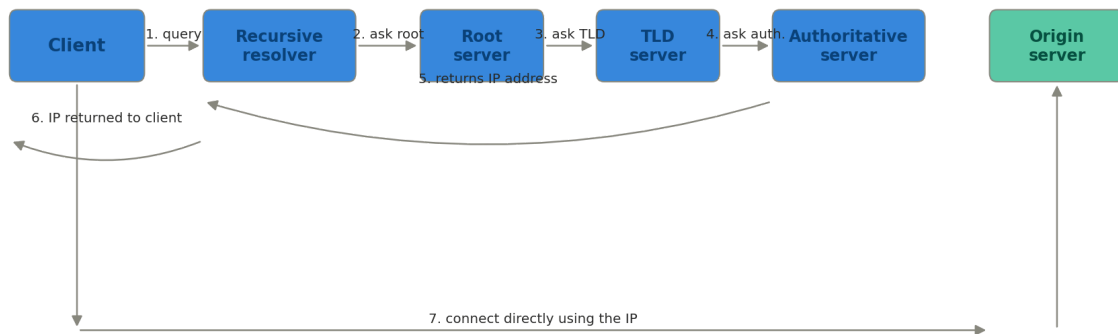
Stage 2 Crash Course: Networking

This is the layer a request physically travels through before your code ever runs. You can't reason about latency budgets or failure domains without knowing what's between the client and your service.

1. DNS Resolution

Domain Name System: translates a human-readable domain (api.example.com) into an IP address a computer can actually connect to.

Why it exists: IP addresses are how machines route traffic, but they're not memorable or stable (a service can move to a new IP without you needing to change anything client-side) — DNS is the indirection layer that makes that possible.



The resolution path, step by step:

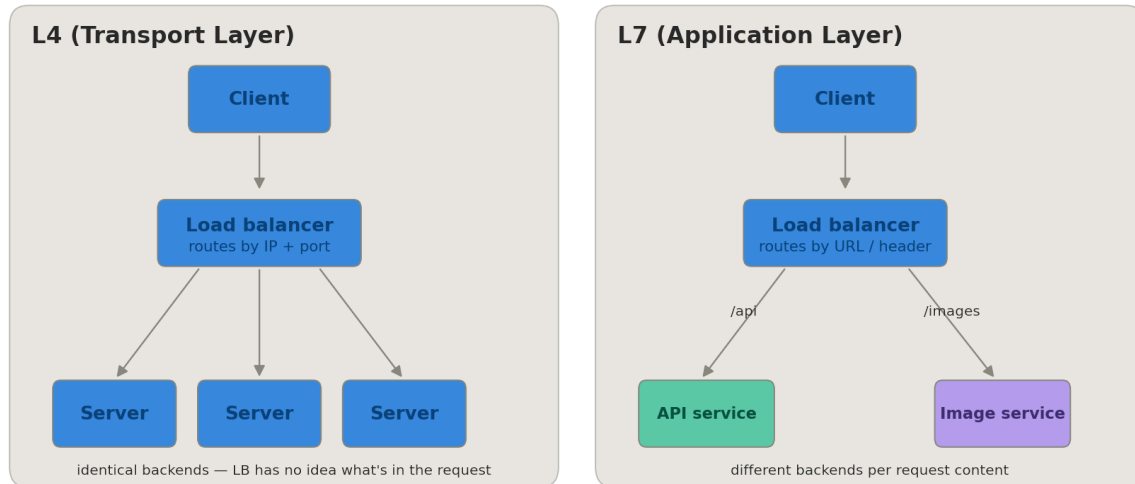
1. Client asks its configured recursive resolver (often your ISP or a public one like 8.8.8.8) to resolve the domain.
2. The resolver asks a root server, which doesn't know the answer but knows which TLD server (.com, .org, etc.) to ask next.
3. The TLD server doesn't know the final answer either — it refers the resolver to the domain's authoritative server.
4. The authoritative server holds the actual DNS records and returns the IP address.
5. That answer is cached at the resolver (respecting a TTL) and returned to the client.
6. The client connects directly to the origin server using the resolved IP — DNS is no longer involved in the actual request/response.

Why TTL matters: a low TTL means changes propagate fast but you pay more resolution overhead; a high TTL is cheaper but a DNS change (e.g., failover to a backup region) takes longer to actually take effect everywhere.

2. Load Balancers

A load balancer distributes incoming requests across multiple backend servers so no single server is overwhelmed, and so a dead server can be silently routed around.

L4 vs. L7



- L4 (transport layer): routes based on IP address and port only — it never looks at the actual HTTP request. Faster, protocol-agnostic, but can't make content-aware decisions.
- L7 (application layer): reads the actual request — URL path, headers, cookies — and can route `/api/*` to one service and `/images/*` to another, or route based on a user's session cookie for sticky sessions.

When to use which: L4 when you just need raw throughput and don't care what's inside the request (e.g., balancing raw TCP connections); L7 when routing decisions depend on the request's content, which is most modern web/microservice traffic.

Routing algorithms

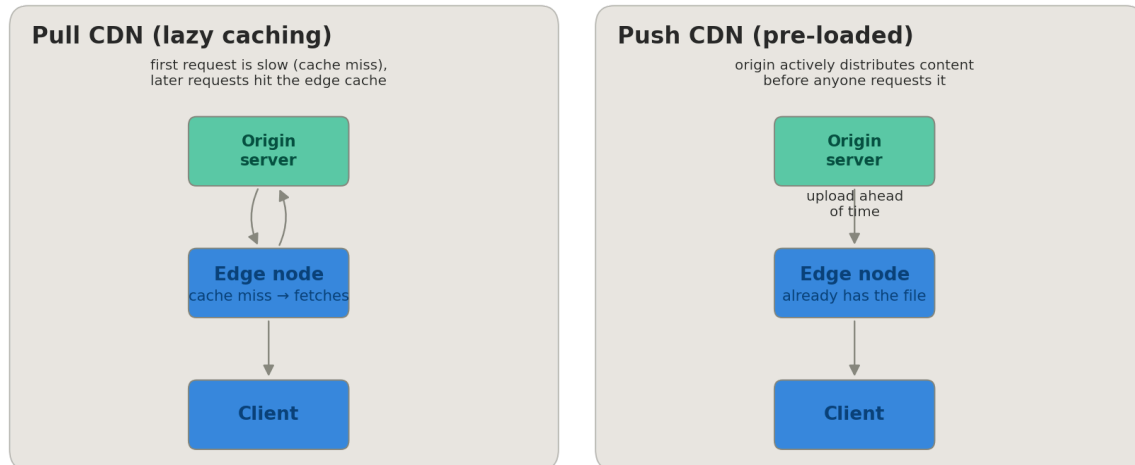
- Round robin: cycles through servers in order — simple, assumes all servers and all requests are roughly equal cost.
- Least connections: sends the next request to whichever server currently has the fewest active connections — better when request cost varies a lot.
- Consistent hashing: routes based on a hash of some request property (e.g., user ID) so the same client tends to land on the same server — useful for cache locality.

Health checks

The load balancer periodically pings each backend (e.g., a `/health` endpoint). A server that fails checks is pulled out of rotation automatically — this is the actual mechanism that turns "redundancy" from a theoretical property into something that works during a real failure.

3. CDNs (Content Delivery Networks)

A CDN is a globally distributed set of edge servers that cache content physically close to users, so requests don't have to round-trip all the way to your origin server.



- Pull (lazy): the edge node fetches from the origin on the first request (cache miss) and caches it for subsequent requests. Simple, but the first request per region is slow.
- Push: the origin proactively uploads content to edge nodes ahead of demand. No cold-miss penalty, but requires knowing in advance what needs to be distributed — works well for a known set of static assets, not for arbitrary user-generated content.

Edge invalidation: when content changes at the origin, every edge copy is now stale until it's invalidated or its TTL expires. This is the same cache-invalidation problem you'll see again in Stage 4, just distributed across the globe instead of across one data center.

Important limitation: a CDN helps enormously with latency for cacheable, mostly-static content (images, JS bundles, video). It does essentially nothing for a write-heavy API or personalized dynamic content that can't be cached at the edge.

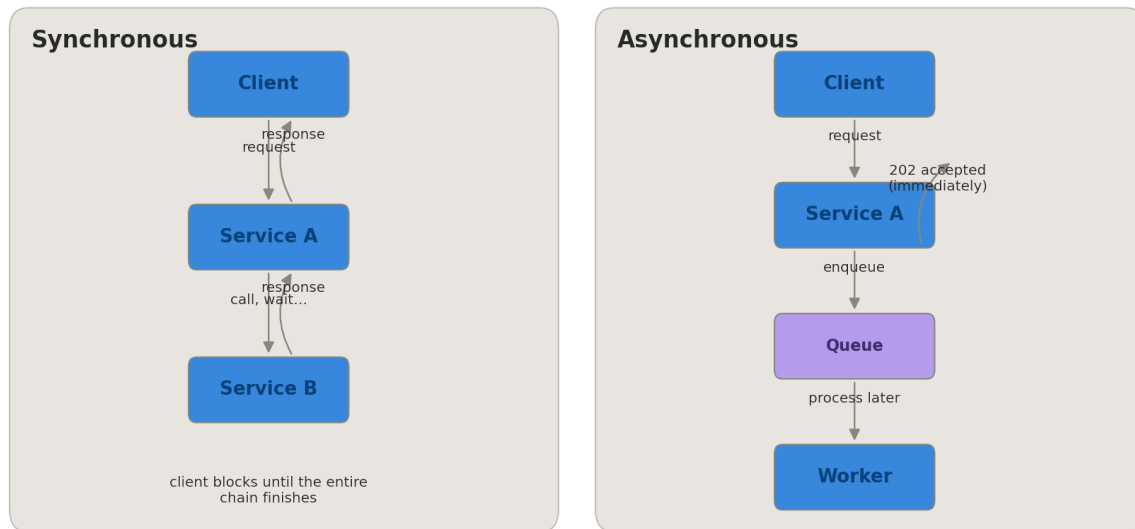
4. REST vs. GraphQL vs. gRPC vs. WebSockets

These solve different shapes of the same underlying problem: how a client and server exchange structured data.

	REST	GraphQL	gRPC	WebSockets
Data format	JSON over HTTP	JSON, client-defined query	Binary (Protocol Buffers)	Any, bidirectional
Best for	Simple, cacheable resource APIs	Clients needing flexible, varying data shapes	Low-latency internal service-to-service calls	Real-time, persistent connections (chat, live updates)
Weakness	Over/under-fetching data	Complex caching, N+1 query risk	Not human-readable, weak browser support	Stateful connection is harder to load-balance/scale
Typical use	Public web APIs	Mobile apps with varied screens/data needs	Microservice-to-microservice	Chat apps, live dashboards, multiplayer

Decision framing: REST is the default for a public-facing resource API. GraphQL earns its complexity when different clients (web vs. mobile) genuinely need different slices of the same data. gRPC earns its complexity for internal service-to-service calls where performance matters and you control both ends. WebSockets are the only option among these four for true server-initiated, persistent, bidirectional communication.

5. Synchronous vs. Asynchronous Communication



- Synchronous: the client blocks and waits for the entire call chain to finish before getting a response. Simple to reason about, but the client's response time is the sum of every downstream call's response time — and a slow or failing downstream service directly becomes the client's problem.
- Asynchronous: the client gets an immediate acknowledgment (e.g., HTTP 202 Accepted) while the actual work happens later via a queue and a worker. The client is decoupled from downstream latency and failures, at the cost of needing a way to know when the work is actually done (polling, webhook, websocket push).

The trap: "just make it async" is not automatically a fix. If work is being produced faster than workers can consume it, you haven't removed the bottleneck — you've just moved it into a growing queue, which becomes its own latency and reliability problem (see Stage 5, backpressure). Async is the right call when the client genuinely doesn't need the result immediately, not a way to avoid capacity planning.

Must be able to explain before moving on

7. Why a low DNS TTL trades faster propagation for higher resolution overhead.
8. A concrete scenario where you'd need L7 routing and L4 genuinely couldn't do the job.
9. Why a CDN does nothing for a write-heavy, highly personalized API.
10. Which of REST/GraphQL/gRPC/WebSockets you'd pick for: a public weather API, a mobile app with very different iOS/Android data needs, two internal services on the same backend team, and a live multiplayer game — and why.
11. Why "just make it async" can hide a capacity problem instead of solving it.